

Le rendu doit être dactylographié et au format PDF, les autres formats ne seront pas corrigés.

Pour les questions commençant par “*Présenter une solution*”, vous devez expliquer textuellement comment fonctionne votre solution. Pour les questions commençant par “*Donner un algorithme*”, vous devez donner l’algorithme, avec l’explication de son fonctionnement et des commentaires.

Pour les algorithmes, vous ne disposez pas d’autres objets/fonctions que des listes, files ou piles et leurs fonctions associées définies dans le cours.

Attention, une réponse ne sera pas notée si:

- Le soin apporté à la rédaction et à la copie sont insuffisants,
- Toutes les réponses ne sont pas justifiées par un raisonnement clair et précis ou expliquées.

Exercice 1 : Balles rebondissantes (10 points)

Nous disposons de balles rebondissantes qui, lancées contre un mur, rebondissent en direction de la personne qui lance. En fonction de sa distance au mur p , en nombre de pas, la personne peut rattraper la balle ou non. En dessous d’un nombre de pas $p_{\max}$, la balle rebondit suffisamment pour pouvoir être rattrapée et réutilisée. Au delà de $p_{\max}$ la balle ne rebondit plus suffisamment pour pouvoir être rattrapée, elle est perdue. La valeur de $p_{\max}$ est la même pour toutes les balles et la taille de la pièce est limitée à P pas, donc $0 \leq p_{\max} \leq P$. Notre objectif est de trouver la valeur de $p_{\max}$ en un nombre minimal d’essais e .

Question 1.1 : *Proposer une stratégie permettant de trouver $p_{\max}$ avec une balle. Justifiez votre réponse.*

Question 1.2 : *Quel est le nombre minimum d’essais nécessaires pour être sûrs de trouver $p_{\max}$ dans une pièce de taille P avec une balle?*

Attention, nous devons être sûr de trouver $p_{\max}$. Nous ne pouvons donc pas faire un lancé qui ne permettrait pas de recevoir la balle s’il ne permet pas d’en déduire $p_{\max}$ puisque après nous n’aurions plus de balle pour tester d’autres valeurs de p .

Question 1.3 : Proposer une stratégie permettant de trouver $p_{\max}$ avec deux balles lorsque la taille de la pièce est $P = 10$. Quel est le nombre minimal d'essais e nécessaires pour trouver à coup sûr $p_{\max}$?

Nous nous intéressons maintenant au problème général, avec un nombre de balles n et une taille de pièce P . Dans ce cas, trouver directement $p_{\max}$ est difficile mais il est possible, pour un nombre d'essais e et un nombre de balles n , de trouver la taille maximale de la pièce P dans laquelle nous pouvons trouver $p_{\max}$. Pour cela nous partons d'une distance d pour faire le premier essai. Si la personne rattrape la balle cela signifie que la distance d est telle que $d \leq p_{\max} \leq P$. Il reste alors n balles mais plus que $e - 1$ essais possibles. Si la personne ne rattrape pas la balle cela signifie que $p_{\max} < d$ et il reste alors $n - 1$ balles mais plus que $e - 1$ essais possibles.

Question 1.4 : Donner la relation de récurrence qui permet de trouver la taille maximale P de la pièce dans laquelle il est possible de trouver $p_{\max}$.

Question 1.5 : Donner un algorithme de programmation dynamique retournant la taille maximale P de la pièce pour n balles et e essais.

Question 1.6 : En déduire une stratégie permettant de trouver à coup sûr la valeur de $p_{\max}$ dans une pièce de taille P , avec n balles.

Exercice 2 : Plus petit cycle dans un graphe) (5 points)

Soit un graphe $G = (V, E)$ orienté quelconque sans pondération sur ses arcs. Dans cet exercice nous cherchons à trouver le plus petit cycle du graphe G .

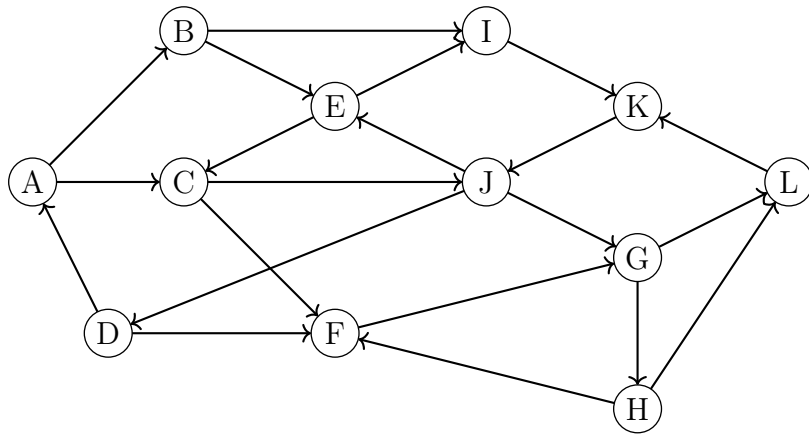


Figure 1: Graphe $G = (V, E)$ orienté sans pondération sur ses arcs

Question 2.1 : Proposer une solution pour trouver le plus petit cycle dans le graphe donné à la figure 1: illustrer votre proposition à partir du graphe et en déroulant celle-ci.

Question 2.2 : Écrire sous forme algorithmique la fonction $ppCycle(G[])$ qui prend en paramètre un graphe décrit sous la forme d'un tableau G de listes de voisins et retourne une liste C constituée des sommets qui composent le plus petit cycle.

Question 2.3 : Cet algorithme est-il optimal? Prouver votre réponse.

Exercice 3 : Liste de commandes (5 points)

Une entreprise doit réaliser une liste de commandes c_i . Chacune de ces commandes prend le même temps à être réalisée et l'entreprise ne peut réaliser qu'une seule commande à la fois. Les dates d_i auxquelles les commandes doivent être finies au plus tard ne sont pas les mêmes et les contrats des commandes prévoient qu'une pénalité p_i est à payer en cas de retard sur la réalisation de la commande. Comme le gain par commande est le même, l'entreprise cherche à minimiser les pénalités qu'elle aura à payer si elle ne peut pas finir certaines commandes à temps.

c_i	1	2	3	4	5	6	7	8
d_i	2	2	2	3	4	5	6	6
p_i	4	3	1	3	2	3	2	1

Table 1: Liste de commandes

Question 3.1 : Donner un ordonnancement des commandes données dans le tableau 1 qui minimise les pénalités de retard.

À noter que, puisque toutes les commandes ont la même durée, le problème est équivalent au cas où toutes les commandes ont une durée de 1. Les commandes données dans le tableau ont donc une durée unitaire.

Question 3.2 : Proposer une solution gloutonne pour permettre de choisir la liste de commandes CR à réaliser, parmi un tableau de n commandes $C[]$, pour minimiser les pénalités.

Question 3.3 : Donner l'algorithme qui implémente votre solution

Pour obtenir la date limite de la commande i vous pouvez utiliser la notation $C[i].d$, de la même manière vous pouvez utiliser la notation $C[i].p$ pour obtenir la pénalité associée. De plus, vous pouvez supposer l'existence de deux fonctions: $comParD(C, L)$ et $comParP(C, L)$ qui rendent une liste triée des numéros des commandes contenus dans la liste L , respectivement en fonction des dates limites et des pénalités.

Question 3.4 : Votre solution est-elle optimale? Prouvez votre réponse.